Умовні. Приклади

На площині заданий трикутник ABC. Знайти його площу. Перевірити дані на коректність.

```
xA = float(input('xA = '))
yA = float(input('yA = '))
xB = float(input('xB = '))
yB = float(input('yB = '))
xC = float(input('xC = '))
yC = float(input('yC = '))

# Перевіримо, чи не лежать усі три точки на одній прямій
from math import sqrt # square root
AB = sqrt((xA-xB)**2 + (yA-yB)**2)
AC = sqrt((xA-xC)**2 + (yA-yC)**2)
BC = sqrt((xB-xC)**2 + (yB-yC)**2)
if AB + BC == AC або AB + AC == BC або BC + AC == AB:
    print('Точки лежать на одній прямій')
else:
    # Знаходимо площу за формулою Герона
    p = 0.5 * (AB + AC + BC)
    S = sqrt(p * (p - AB) * (p - AC) * (p - BC))
    print('S =', S)
```

Цикли

Оператори циклів використовують у разі, якщо у програмі необхідно багаторазово повторити деяку послідовність операторів. У мові Python існує два таких операторів: ***for*** і ***while***. Оператор for використовується у випадках, коли кількість повторів циклу заздалегідь відомо, а інших випадках використовуються оператори while.

Оператор for

В операторі ***for*** кількість повторів задається за допомогою змінної, яка називається параметром циклу. Існує дві форми запису оператора ***for***. Перша форма використовується у тому випадку, коли значення параметра циклу при кожному повторі зростає:

```
for параметр циклу in range(([start,] stop[, step])):
блок_операторів;
```

Оператор for виконується до тих пір, поки значення параметра циклу не досягне кінцевого значення у списку

for i in range(10): print(i)	for i in range(1,10,3): print(i)	for i in range(2,10,2): print(i)	for i in range(10,0,-1): print(i)
1 2 3 4 5	1 4 7	2 4 6 8	10 9 8 7 6

6 7 8 9			5 4 3 2 1

Приклад. Знайдемо суму n чисел ряду

```
n=int(input("Введіть число членів суми 0+1+...+n "))
s=0 # нейстральний елемент відносно операції додавання
for i in range(n):
    s=s+i
print(s)
```

Приклад. Найменший дільник (https://www.eolymp.com/ru/problems/8927)

Для заданого натурального числа n виведіть найменший дільник, відмінний від 1.

```
x=int(input());
import math
flag=True;
for i in range(2,int(math.sqrt(x))+1):
        if (x%i)==0:
                print(i);
                flag=False;
                break;
if flag: print(x);
```

Приклад. Написати програму обчислення факторіалу

```
n=int(input("Введите n "))
p=1
for i in range(1,n+1):
   p=p*i
print("{0}!={1}".format(n,p)).
5!=120
```

Як перевірити, що число має цілий корінь?

Приклад Табулювання функції

```
a=float(input("Ввести a"));
b=float(input("Ввести b"));
n=100;
h=(b-a)/n;
import  math;
#min =1000;
for i in range(n+1):
  x=a+h*i;
  print(x, math.sin(x));
```

Приклад. Пошук min функції на проміжку

Приклад. Знайдемо суму ряду.

- $\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots = 1 + \sum_{n=1}^{\infty} x^n$, для всех $|x| < 1$

Варіант 1

```
s=0;
x=float(input());
for i in range(0,10):
  s=s+x**i;
print(s);
```

Варіант 2.# зменшимо кількість операцій за рахунок зберігання проміжного значення в змінній xx

```
s=0;
x=float(input());
xx=1;
for i in range(0,10):
  #s=s+x**i;
  s=s+xx;
  xx=xx*x;
print(s);
```

- Експонента: $e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{n=0}^{\infty} \frac{x^n}{n!}, x \in \mathbb{C}$

```
s=0;
x=float(input());
xx=1;
for i in range(0,10):
  #s=s+x**i
  factorial=1
  for j in range(1,i+1):
    factorial=factorial*j
  s=s+xx/factorial;
  xx=xx*x;
print(s);

s=0;
for i in range(10):
        x=int(input())
        if x==-1:
                break
        else:
                s=s+x
print(str(s)+"end")
```

Оператор ***break*** – служить для виходу із циклу

```
for i in range(10):
        if i<6: print(i)
        else: break
print("конец")
```

Інкремент та декремент у Python.

Інкремент – це збільшення значення змінної на одиницю, а декремент – зменшення. На відміну від інших мов програмування у Python немає спеціальних операторів. Проте можна більш компактно записати операцію збільшення чи зменшення значення змінної, тобто замість

x=x+y – пишуть x+=f

x=x-y пишуть x-=y

```
x=10                y=20
x+=5                y-=5
print(x)            print(y)
```

Зауваження. Нехай x=1, додамо до x 0.1, а потім знову 0,1>>> x=1

```
>>> x=x+0.1
>>> x
1.1
>>> x=x+0.1
>>> x
1.2000000000000002
```

Виникнення двійки в молодшому розряді пов'язане з поданням чисел типу float в Python. Під речові числа в Python відводиться 53 двійкових розряду, крім знакового. Тому подання числа після коми у двоїчному коді відбувається поки вистачає розрядів. При зворотному переведенні в десяткову систему числення може вийти число з невеликою похибкою. Яка, наприклад, при порівнянні чисел може не мати значення.

```
for i in range(1,5):
  for j in range(1,5):
    print("a"+str(i)+str(j)," ",end="")
  print()
```